consequence is that the memory model should define which memory locations a store instruction writes to.

For example, what does the code below print? Or, alternatively, can the assignment `p[6] = 0` change any byte of the object pointed to by `q`?

```
char *p = malloc(4);
char *q = malloc(4);
q[2] = 0;
p[6] = 1;
print(q[2]); // prints 0 or 1?
```

In a *flat* memory model, the program would print `1` if `q == p + 4`, and `0` otherwise. A flat memory model treats pointers like integers: a memory-accessing instruction can access any (unprotected) location in memory, and therefore the program is allowed to guess the location of objects (as shown in Figure 1). Some assembly languages have a flat memory model; others, such as those for machines with segmented memory, do not.

While a flat memory model is conceptually simple and is a good match for low-level programming, it hinders high-level optimizations that are routinely performed by and considered essential in modern compilers.

### 2.2 Data-Flow Provenance Tracking

In the previous example, we showed that the program can print either `0` or `1` depending on where the memory allocator places the allocated blocks. This dependence on the run-time behavior of the allocator overconstrains the compiler, blocking it from performing important optimizations such as store forwarding. For example, we want the compiler to be able to propagate the store `q[2] = 0` to the print instruction. Hence, the memory model needs a way to prevent the store to `p[6]` from accessing `q[2]` regardless of where `p` and `q` end up pointing at run time. For example, rules to this effect have been a part of C since C89.

Data-flow provenance tracking provides a way to prevent objects from being accessed via pointers derived from unrelated objects. The idea is that each pointer is a pair of two values: the object to which it can point to, and the memory address (or an offset within that object). It is undefined behavior (UB) to try to access memory with a pointer that is out-of-bounds of its object. This semantics is sufficient to allow the compiler to conclude that `p[6]` cannot access `q[2]`, regardless of the fact that at runtime they may end up referring to the same location.

Data-flow provenance tracking could be defined like this:

```
char *p = malloc(4);   // (val=0x10, obj=p)
char *q = malloc(4);   // (val=0x14, obj=q)
char *q2 = q + 2;      // (val=0x16, obj=q)
char *p6 = p + 6;      // (val=0x16, obj=p)

*q2 = 0;  // OK
*p6 = 1;  // UB, since out-of-bounds of obj p
print(*q2);  // can be replaced with print(0);
```

The first store through `q2` succeeds, since it is within the bounds of object `q`. The second store, however, triggers UB because the pointer is out-of-bounds of its base object (`p`), even though the program correctly guessed the address of a valid object (as shown in Figure 2). Finally, the compiler can safely propagate the store `*q2 = 0` to the print instruction since there are no well-defined store instructions in between.